function Cf_Init takes two parameters: Control Port and Data Port. The question is, which is which? The function prototype `voidCf_Init(char *ctrlport, char *dataport);` makes it clear. If a header file contains function prototypes, you can that file to get the information you need for writing programs that call those functions. If you include an identifier in a prototype parameter, it is used only for any later error messages involving that parameter; it has no other effect.

Function definition

Function definition consists of its declaration and a function body. The function body is technically a block – a sequence of local definitions and statements enclosed within braces {}. All variables declared within function body are local to the function, i.e. they have function scope.

The function itself can be defined only within the file scope. This means that function declarations cannot be nested.

To return the function result, use the return statement. Statement return in functions of void type cannot have a parameter – in fact, you can omit the return statement altogether if it is the last statement in the function body.

Here is a sample function definition:

```
/* function max returns greater of its 2 arguments: */
int max(int x, int y) {
  return (x>=y) ? x : y;
}
```

```
Here is a sample function which depends on side effects rather than return value:
/* function converts Descartes coordinates (x,y) to polar (r,fi): */
#include <math.h>
void polar(double x, double y, double *r, double *fi) {
  *r = sqrt(x * x + y * y);
  *fi = (x == 0 && y == 0) ? 0 : atan2(y, x);
  return; /* this line can be omitted */
}
```

Functions reentrancy

Limited reentrancy for functions is allowed. The functions that don't have their own function frame (no arguments and local variables) can be called both from the interrupt and the "main" thread.